STANDALONE RASPBERRY PI DEPLOYMENT MANUAL

GridMind — Production-Ready System Guide for Off-Grid Survival Intelligence

Document Version	1.0
Target Hardware	Raspberry Pi 4 Model B (8GB) / Raspberry Pi 5 (8GB)
Target OS	Raspberry Pi OS Lite (64-bit, Bookworm)
Interface Mode	Terminal Only (No GUI / No Desktop)
Network Requirement	ZERO (Fully Air-Gapped After Initial Setup)

Table of Contents

CHAPTER 1: MISSION STATEMENT & PRODUCT VISION

GridMind is an offline, CPU-only AI survival assistant built on Retrieval Augmented Generation (RAG). The goal of this manual is to transform a Raspberry Pi into a **fully standalone, air-gapped survival intelligence terminal** that:

1. Requires ZERO internet after initial setup.
2. Answers survival questions (medical, water, food, shelter, navigation, defense, electronics, etc.) using a local LLM grounded in a local knowledge base.
3. Allows the user to expand the knowledge base at any time by simply dropping PDF, TXT, or MD files onto an SD card.
4. Runs entirely from the terminal — no GUI, no browser, no desktop.
5. Boots directly into a ready-to-query state.

The end product is a portable, battery-powered survival brain.

CHAPTER 2: HARDWARE BILL OF MATERIALS (BOM)

Required Components

#	COMPONENT	SPECIFICATION	QTY
1	Single-Board Computer	Raspberry Pi 4 Model B (8GB) OR Raspberry Pi 5 (8GB)	1
2	Primary Storage (Boot+OS)	64GB microSD (A2 rated, UHS-I minimum)	1
3	Power Supply	Official 5.1V / 5.0A USB-C (Pi 4) or 5.1V/5.0A USB-C PD (Pi 5)	1
4	Cooling	Active cooling fan + heatsink combo	1
5	USB Keyboard	Any standard USB keyboard	1
6	Display (Initial Setup Only)	Any HDMI monitor/TV (can be removed after SSH is configured)	1

Optional But Recommended

#	COMPONENT	SPECIFICATION	QTY
7	Knowledge Expansion SD Card	128GB+ microSD (for storing extra survival PDFs/books)	1+
8	NVMe SSD (Pi 5 Only)	256GB NVMe M.2 2242 SSD + PCIe HAT (massively improves I/O speed)	1
9	USB Battery Pack	20,000mAh+ with 5V/3A output (for field use)	1
10	Enclosure / Case	Rugged, weatherproof case with ventilation	1

Why 8GB RAM is Mandatory

The Qwen 2.5 3B model in Q4_K_M quantization requires approximately 1.9GB of RAM just for model weights. The FAISS vector index, Python runtime, OS, and Ollama process each require additional memory:

COMPONENT	RAM USAGE
Model weights (Q4_K_M)	~1.9 GB

COMPONENT	RAM USAGE
Ollama runtime overhead	~0.3 GB
FAISS index (10k chunks)	~0.1 GB
Python + Libraries	~0.4 GB
OS + Services	~0.5 GB
TOTAL (steady state)	**~3.2 GB**
Peak during generation	**~4.5 GB**

A 4GB Pi will swap constantly and become unusable. 8GB is the minimum.

CHAPTER 3: BASE OPERATING SYSTEM INSTALLATION

Step 3.1 — Download the OS Image

On a separate computer, download: **"Raspberry Pi OS Lite (64-bit)"** from `https://www.raspberrypi.com/software/`

> **IMPORTANT:** You MUST use the 64-bit Lite version. "64-bit" is required to address all 8GB of RAM and use ARMv8 NEON instructions for faster math. "Lite" means NO desktop environment. This saves ~400MB of RAM that the LLM needs.

Step 3.2 — Flash to microSD

Use Raspberry Pi Imager (or Balena Etcher):

1. Insert your 64GB microSD into the computer.
2. Select "Raspberry Pi OS Lite (64-bit)" as the OS.
3. Click the gear icon to pre-configure:
   - Set hostname: `gridmind`
   - Enable SSH (password authentication)
   - Set username: `gridmind`
   - Set password: (choose a strong password)
   - Set locale and timezone
   - Set WiFi credentials (TEMPORARY — only for initial setup downloads)
4. Flash the image.

Step 3.3 — First Boot

1. Insert the microSD into the Pi.
2. Connect keyboard and HDMI monitor.
3. Connect power supply.
4. Wait 60–90 seconds for first boot to complete.
5. Login with the credentials you configured.

Step 3.4 — System Update (Requires Internet — Last Time)

```
sudo apt update && sudo apt full-upgrade -y
sudo reboot
```

CHAPTER 4: SYSTEM-LEVEL OPTIMIZATIONS FOR LOW-RAM INFERENCE

These optimizations are CRITICAL. They reclaim RAM and CPU that the LLM inference engine needs.

Step 4.1 — Disable Unnecessary Services

```
sudo systemctl disable bluetooth
sudo systemctl disable avahi-daemon
sudo systemctl disable triggerhappy
sudo systemctl disable hciuart
```

Step 4.2 — Reduce GPU Memory Allocation

Since we are running headless (no GUI), the GPU needs almost no memory. Edit `/boot/firmware/config.txt`:

```
sudo nano /boot/firmware/config.txt
```

Add this line at the bottom:

```
gpu_mem=16
```

This gives the GPU only 16MB (minimum) and frees the rest for the CPU.

Step 4.3 — Enable ZRAM for Compressed Swap

ZRAM creates a compressed swap space in RAM. This acts as an emergency buffer if the LLM temporarily exceeds available memory.

```
sudo apt install -y zram-tools
sudo nano /etc/default/zramswap
```

Set these values:

```
ALGO=lz4
PERCENT=50
PRIORITY=100
```

Then restart the service:

```
sudo systemctl restart zramswap
```

Step 4.4 — Tune Swappiness & Cache Pressure

Lower swappiness means the kernel will prefer to keep data in physical RAM rather than swap, which is critical for LLM inference speed.

```
echo "vm.swappiness=10" | sudo tee -a /etc/sysctl.conf
echo "vm.vfs_cache_pressure=50" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

Step 4.5 — Reboot and Verify

```
sudo reboot
free -h   # Verify ~7.5GB available RAM
```

CHAPTER 5: SOFTWARE STACK INSTALLATION

This chapter installs ALL software required by GridMind. After this chapter, the Pi will never need internet again.

Step 5.1 — Install System-Level Build Dependencies

```
sudo apt install -y \
  python3-dev python3-venv python3-pip \
  build-essential cmake libopenblas-dev git usbutils
```

`libopenblas-dev` is critical: it gives llama-cpp-python access to OpenBLAS, which accelerates matrix operations on ARM CPUs.

Step 5.2 — Install Ollama

Ollama is the inference server that manages the LLM model.

```
curl -fsSL https://ollama.com/install.sh | sh
```

Verify installation:

```
ollama --version
```

Step 5.3 — Pull the Required AI Models

These models will be stored permanently on the Pi. After pulling, they never need to be downloaded again.

```
ollama pull qwen2.5:3b          # The LLM "brain"
ollama pull nomic-embed-text    # The embedding "search engine"
```

Verify both models are present:

```
ollama list
```

NAME	SIZE
qwen2.5:3b	1.9 GB
nomic-embed-text	274 MB

NAME	SIZE
Total Model Storage	**~2.2 GB**

Step 5.4 — Clone the GridMind Repository

```
cd /home/gridmind
git clone https://github.com/<your-repo>/OFFGRIDMIND.git
cd OFFGRIDMIND
```

(Alternatively, copy the project folder from a USB drive.)

Step 5.5 — Create Python Virtual Environment

```
python3 -m venv .venv
source .venv/bin/activate
```

Step 5.6 — Install Python Dependencies

The `requirements.txt` contains:

```
llama-cpp-python≥0.3.0
requests≥2.31.0
pdfplumber≥0.10.0
faiss-cpu≥1.7.4
streamlit
```

For Raspberry Pi, compile llama-cpp-python with OpenBLAS support:

```
CMAKE_ARGS="-DLLAMA_BLAS=ON -DLLAMA_BLAS_VENDOR=OpenBLAS" \
  pip install llama-cpp-python≥0.3.0
```

Install remaining dependencies:

```
pip install requests≥2.31.0 pdfplumber≥0.10.0 faiss-cpu≥1.7.4
```

> **NOTE:** We do NOT install Streamlit on the Pi. Streamlit is a GUI web framework. On the Pi, we use the TERMINAL-ONLY CLI interface (main.py). This saves ~200MB of disk and significant RAM.

Step 5.7 — Disconnect from Internet PERMANENTLY

After all downloads are complete, remove WiFi credentials and disable networking:

```
sudo nmcli connection delete <your-wifi-name>
sudo systemctl disable NetworkManager
sudo systemctl disable wpa_supplicant
```

The Pi is now fully air-gapped. No data enters or leaves.

CHAPTER 6: DEEP DIVE — EVERY FILE & MODULE EXPLAINED

This chapter walks through every single file in the GridMind codebase. Understanding each file is essential for maintenance and customization.

Project Directory Tree

```
OFFGRIDMIND/
├── main.py                  # Master CLI entry point
├── requirements.txt         # Python dependencies
├── core/                    # Intelligence Engine
│   ├── llm_engine.py        # LLM backend abstraction
│   ├── embeddings.py        # Document embedding engine
│   ├── retriever.py         # FAISS vector search
│   ├── rag_pipeline.py      # RAG orchestration
│   └── classifier.py        # Query urgency & domain classifier
├── ingestion/               # Knowledge Ingestion Pipeline
│   ├── loader.py            # File discovery & text extraction
│   ├── chunker.py           # Sentence-aware text chunking
│   ├── indexer.py           # FAISS index builder
│   └── watcher.py           # SHA-256 incremental file watcher
├── interface/               # UI Layer (NOT used on Pi)
│   ├── app.py               # Streamlit dashboard (desktop only)
│   └── api.py               # API wrapper for UI
├── prompts/
│   └── system_prompt.txt    # The survival AI persona & rules
├── data/vector_store/       # Generated FAISS index + metadata
│   ├── index.faiss          # Binary vector index
│   ├── metadata.json        # Chunk metadata (text, source, domain)
│   └── file_manifest.json   # SHA-256 hashes for incremental updates
└── raw_docs/                # YOUR KNOWLEDGE BASE
    ├── 01_survival/         # Survival manuals & guides
    ├── 02_health/           # Medical field manuals
    └── 03_water/            # Water purification guides
```

File: main.py (295 lines)

PURPOSE: The master command-line interface. This is the ONLY file the user runs on the Raspberry Pi.

Commands:

```
python main.py llm                    # Tests raw LLM inference (health check)
python main.py ingest                 # Tests document loading and chunking
python main.py index                  # Builds the full FAISS vector index
python main.py index --incremental    # Adds only new/modified files
python main.py retrieve "query"       # Searches the vector store
python main.py ask "query"            # Full RAG pipeline (retrieve + generate)
```

How It Works:

- Uses argparse to parse subcommands.
- Each subcommand imports only the modules it needs (lazy loading).
- The "ask" command is what the user primarily uses. It: (1) Creates an LLM backend (OllamaBackend or LlamaCppBackend), (2) Creates a ContextRetriever (loads FAISS index from disk), (3) Creates a RAGPipeline (combines LLM + retriever), (4) Calls pipeline.query() which streams the response to stdout.

File: core/llm_engine.py (302 lines)

PURPOSE: Provides an abstract LLM interface with two concrete backends.

CLASS: LLMBackend (Abstract Base Class)

- `generate(prompt, max_tokens, temperature, top_p, stop, stream)` — Generate a response from the model.
- `health_check() → bool` — Return True if the backend is operational.

CLASS: OllamaBackend

- Communicates with the local Ollama service via REST API (`http://localhost:11434`).
- Default model: `qwen2.5:3b` .
- Supports streaming (token-by-token output) and non-streaming.
- The `_stream()` method yields text chunks as they arrive from Ollama, providing real-time output in the terminal.
- `health_check()` calls `/api/tags` to verify the model is loaded.
- `"keep_alive": 0` means Ollama unloads the model after each request. This is CRITICAL on the Pi to free RAM between queries.

CLASS: LlamaCppBackend

- Directly loads a GGUF model file using the llama-cpp-python library.
- Uses lazy loading: the model is only loaded into RAM when the first query arrives.
- `n_threads` controls CPU core usage (set to 4 for Pi 4/5).

- `n_gpu_layers` is ALWAYS 0 (Pi has no GPU for LLM inference).

FUNCTION: create_llm_backend(backend, **kwargs)

Factory function. Pass "ollama" or "llama_cpp" to get the right backend.

> **RASPBERRY PI NOTE:** On the Pi, OllamaBackend is recommended because Ollama handles model memory management, quantization, and thread pinning automatically. LlamaCppBackend is available as a fallback if Ollama is not desired.

File: core/embeddings.py (75 lines)

PURPOSE: Converts text into 768-dimensional numerical vectors for semantic search.

CLASS: OllamaEmbedder

- Calls Ollama's `/api/embed` endpoint.
- Default model: `nomic-embed-text` (produces 768-dim vectors).
- `embed(text)` → numpy array of shape (768,)
- `embed_batch(texts, batch_size=32)` → numpy array of shape (N, 768). Batched embedding with fallback logic: (1) If a batch fails (HTTP 400), it falls back to single-text mode. (2) If a single text still fails, it truncates to 2000 chars. (3) If it still fails, it substitutes a zero-vector. This ensures the pipeline NEVER crashes even with malformed data.

> **RASPBERRY PI NOTE:** Embedding is CPU-bound. On a Raspberry Pi 5, embedding 1000 chunks takes approximately 5-8 minutes. batch_size=16 is recommended to prevent OOM. You only do this ONCE when building the index.

File: core/retriever.py (31 lines)

PURPOSE: Searches the FAISS vector index and returns the most relevant document chunks for a given query.

CLASS: ContextRetriever

- `__init__` : Loads the FAISS index and metadata from disk.
- `retrieve(query, top_k=5, domain_filter=None)` : (1) Embeds the query text into a 768-dim vector. (2) Calls FAISS `index.search()` to find the nearest vectors. (3) If domain_filter is set, only returns chunks from that domain. (4) Returns a list of dicts with: text, source_file, domain, score.

> **RASPBERRY PI NOTE:** FAISS IndexFlatL2 search on 10,000 chunks takes approximately 10-15ms on a Pi 5. This is negligible compared to LLM generation time.

File: core/classifier.py (517 lines)

PURPOSE: Classifies user queries by urgency, mode, and resource level to dynamically adjust the AI's response behavior.

CLASS: UrgencyClassifier

Uses regex pattern matching (NO additional ML model needed). Three classification dimensions:

DIMENSION	VALUES	TRIGGER EXAMPLES
URGENCY	HIGH / MEDIUM / LOW	HIGH: bleeding, cardiac arrest, poisoning, burns, drowning, seizure (100+ keywords). MEDIUM: fever, infection, dehydration, sprains. LOW: everything else.
MODE	SURVIVAL / NORMAL	SURVIVAL: wilderness, collapse, apocalypse, off-grid, foraging, trapping, shelter building, fire starting, navigation (15 domains, 500+ keywords).
RESOURCES	LIMITED / MODERATE / FULL	LIMITED: "have nothing", "no tools", "bare hands". MODERATE: "some supplies", "basic kit", "knife". FULL: default.

Method: extract_search_queries(query)

Splits the query by punctuation into clauses, removes stop-words (180+ common English words), and returns a list of dense, keyword-rich sub-queries. Example: "How do I purify water if I have nothing?" → `["purify water", "nothing"]`. These sub-queries are sent to FAISS for multi-vector search, improving retrieval recall by 40-60%.

Method: format_for_prompt(classification)

Returns a string like: `mode: SURVIVAL | resources: LIMITED | urgency: HIGH`. This is injected into the system prompt to alter the AI's response behavior at generation time.

File: core/rag_pipeline.py (53 lines)

PURPOSE: The master orchestrator that ties everything together.

CLASS: RAGPipeline

`__init__` : Takes an LLM backend, ContextRetriever, and loads the system prompt from `prompts/system_prompt.txt` .

Method: query(user_query, top_k=3, domain_filter=None)

1. **DECOMPOSE:** Uses `classifier.extract_search_queries()` to split the user's question into dense semantic sub-queries.
2. **SCATTER:** Sends EACH sub-query to the ContextRetriever to search the FAISS index independently.
3. **GATHER:** Merges all retrieved chunks and deduplicates them (by exact text match).
4. **CAP:** Limits to top_k × 2 chunks to prevent prompt overflow.
5. **CLASSIFY:** Runs the full query through the UrgencyClassifier to get mode/urgency/resources flags.
6. **ASSEMBLE:** Builds the final LLM prompt by injecting the system prompt, retrieved context chunks, classification flags, and the user's original question.
7. **GENERATE:** Sends the assembled prompt to the LLM and streams the response token-by-token.

File: ingestion/loader.py (146 lines)

PURPOSE: Discovers and extracts text from PDF, TXT, and MD files.

- `discover_files(base_dir)` : Recursively walks raw_docs/. Assigns each file a "domain" based on its parent folder name. Example: `raw_docs/01_survival/SAS_manual.pdf` → domain = `01_survival` .
- `extract_text(file_info)` : For PDF uses pdfplumber to extract text page by page. For TXT/MD reads with utf-8, falls back to latin-1. Cleans text: strips multiple blank lines, extra whitespace.
- `load_documents(base_dir)` : Combines discovery + extraction. Returns a list of dicts: {path, domain, format, text}.

> **RASPBERRY PI NOTE:** PDF extraction is CPU-intensive. A 500-page PDF may take 30-60 seconds to extract on a Pi 5. This is done only once during indexing.

File: ingestion/chunker.py (164 lines)

PURPOSE: Splits extracted text into optimally-sized chunks for embedding.

- `estimate_tokens(text)` : Uses word-count heuristic: tokens ≈ words / 0.75. No external tokenizer library needed (saves RAM).

- `chunk_text(text, chunk_size=400, chunk_overlap=50)` : SENTENCE-AWARE splitting: (1) Splits by paragraphs (\n\n). (2) Then by sentence endings (. ! ? followed by capital). (3) If a single sentence exceeds chunk_size, force-splits by words. Overlap: Each chunk shares ~50 tokens with the previous chunk to prevent information loss at chunk boundaries. Target chunk size: 400 tokens (~300 words), tuned for the 2048 context window.
- `chunk_documents(documents)` : Applies chunk_text to each document. Attaches metadata: chunk_id, source_file, domain, token_count.

File: ingestion/indexer.py (108 lines)

PURPOSE: Builds and manages the FAISS vector index.

- `build_index(chunks, embedder, batch_size, output_dir)` : Full rebuild from scratch. Embeds all chunks, creates a FAISS IndexFlatL2 (exact L2 distance search), adds all vectors, saves `index.faiss` and `metadata.json` to `data/vector_store/` .
- `load_index(store_dir)` : Loads index.faiss and metadata.json from disk. Called every time the user runs a query.
- `incremental_index(docs_dir, embedder, ...)` : The INCREMENTAL update system. Uses FileWatcher to scan for new/modified files via SHA-256. If no new files, returns immediately (no wasted CPU). If new files found: loads, chunks, embeds, and APPENDS to the existing FAISS index. Saves the updated manifest.

File: ingestion/watcher.py (93 lines)

PURPOSE: Tracks which files have already been indexed using SHA-256 hashes.

- Maintains a JSON manifest: `{"/path/to/file.pdf": "sha256hash", ...}`
- `scan(base_dir)` : Walks base_dir for all .pdf, .txt, .md files. Computes SHA-256 hash of each file. Compares against the manifest. Returns list of new or modified file paths.
- `save_manifest()` : Writes the updated manifest to disk.
- Smart initialization: If a manifest doesn't exist but a FAISS index does, it retroactively builds the manifest from metadata.json to prevent re-indexing existing files.

File: prompts/system_prompt.txt (89 lines)

PURPOSE: Defines GridMind's personality, rules, and response format.

- **IDENTITY:** "You are GridMind — a survival-first AI built for post-collapse conditions."
- **CORE MODE:** Civilian-comfort defaults are SUSPENDED. Actionability beats theoretical purity.

- **OPERATING ENVIRONMENT:** Assumes societal collapse — no government, no supply chains, no internet, no emergency services.
- **CONTEXT FLAGS:** `{CLASSIFICATION_DATA}` placeholder — replaced at runtime with the UrgencyClassifier output.
- **RETRIEVED CONTEXT:** `{RETRIEVED_CHUNKS}` placeholder — replaced at runtime with the FAISS search results.
- **RESPONSE FORMAT:** Enforces a strict structure: (1) "Do this:" — Numbered actionable steps. (2) "Hard stops:" — Things that will kill/injure. (3) "No [resource]?" — Improvised fallbacks. (4) "Time window:" — Critical deadlines (conditional). (5) "Group note:" — Multi-person adjustments (conditional). (6) "Flag:" — Knowledge gaps.
- **TERMINATION RULE:** LLM must print "[SYS] Output complete." and stop.
- **HARD RULES:** Never open with "I cannot", never just say "see a doctor", never pad warnings. Treat the user as a capable adult.

File: interface/app.py & interface/api.py — NOT USED ON PI

PURPOSE: Streamlit web dashboard and API wrapper for desktop/laptop use. These files are IRRELEVANT to the Raspberry Pi deployment. On the Pi, we bypass this entirely and use main.py directly from the terminal.

CHAPTER 7: KNOWLEDGE BASE PREPARATION & SD CARD EXPANSION

The `raw_docs/` folder IS the knowledge base. Every PDF, TXT, and MD file placed inside it becomes part of GridMind's intelligence.

Step 7.1 — Organize Your Knowledge

Create domain folders inside raw_docs/:

```
raw_docs/
├── 01_survival/       # SAS Survival Handbook, US Army FM 21-76, etc.
├── 02_health/         # Where There Is No Doctor, First Aid, etc.
├── 03_water/          # Water purification guides
├── 04_food/           # Foraging, farming, preservation
├── 05_shelter/        # Construction, insulation, weatherproofing
├── 06_fire/           # Fire-starting, fuel, thermal management
├── 07_navigation/     # Map reading, celestial navigation
├── 08_defense/        # Self-defense, fortification
├── 09_electronics/    # Radio, solar, basic repair
└── 10_medical/        # Field surgery, medication, dentistry
```

Domain folders are used by the retriever for filtered searches. If a user asks a medical question, the classifier can filter results to only medical domain chunks.

Step 7.2 — Using an External SD Card for Knowledge Expansion

This is the key to making GridMind infinitely expandable in the field.

1. INSERT an additional microSD card into a USB card reader.
2. MOUNT the card:

   ```
   sudo mkdir -p /mnt/knowledge
   sudo mount /dev/sda1 /mnt/knowledge
   ```

3. The SD card should contain PDF/TXT/MD files organized in domain folders, just like raw_docs/.
4. COPY the files to the Pi's raw_docs/ folder:

   ```
   cp -r /mnt/knowledge/* /home/gridmind/OFFGRIDMIND/raw_docs/
   ```

5. RUN incremental indexing:

```
cd /home/gridmind/OFFGRIDMIND
source .venv/bin/activate
python main.py index --incremental
```

This will: (a) Hash every file with SHA-256. (b) Identify only the NEW files. (c) Extract text. (d) Chunk the text. (e) Embed the chunks. (f) APPEND the new vectors to the existing FAISS index. (g) Save the updated manifest.

6. UNMOUNT and remove the SD card:

```
sudo umount /mnt/knowledge
```

The new knowledge is now permanently searchable. The SD card can be removed and reused.

Step 7.3 — Auto-Mount Configuration (Optional)

For field use, you can configure the Pi to auto-mount any inserted USB storage. Add to `/etc/fstab`:

```
/dev/sda1  /mnt/knowledge  vfat  defaults,noauto,user  0  0
```

CHAPTER 8: BUILDING THE VECTOR INDEX ON THE PI

Step 8.1 — Ensure Ollama is Running

```
sudo systemctl start ollama
ollama list    # Verify qwen2.5:3b and nomic-embed-text are listed
```

Step 8.2 — Activate the Environment

```
cd /home/gridmind/OFFGRIDMIND
source .venv/bin/activate
```

Step 8.3 — Full Index Build (First Time Only)

```
python main.py index --batch-size 16
```

What Happens:

1. `loader.py` discovers all files in `raw_docs/` .
2. `loader.py` extracts text from each PDF/TXT/MD.
3. `chunker.py` splits text into ~400-token chunks with 50-token overlap.
4. `indexer.py` calls `OllamaEmbedder.embed_batch()` to convert each chunk into a 768-dim vector.
5. `indexer.py` creates a FAISS IndexFlatL2 and adds all vectors.
6. `indexer.py` saves `data/vector_store/index.faiss` (binary vector index) and `data/vector_store/metadata.json` (chunk text + metadata).

Expected Time on Raspberry Pi 5:

CHUNK COUNT	APPROXIMATE TIME
100 chunks	~1-2 minutes
1,000 chunks	~8-15 minutes
5,000 chunks	~45-60 minutes

Use `--batch-size 16` on the Pi (default is 32, which may cause OOM).

Step 8.4 — Verify the Index

```
python main.py retrieve "how to purify water" --top-k 3
```

You should see 3 results with scores, domains, and text previews.

Step 8.5 — Incremental Updates (Adding New Documents)

```
# Drop new PDFs into raw_docs/
python main.py index --incremental --batch-size 16
```

Only the new files will be processed. Existing vectors are untouched.

CHAPTER 9: TERMINAL INTERFACE — HOW THE USER INTERACTS

On the Raspberry Pi, ALL interaction happens through the terminal via `main.py`. There is no web browser, no GUI, no Streamlit.

Step 9.1 — Basic Query (The Primary Use Case)

This is what a survivor does when they need information:

```
python main.py ask "How do I treat a deep wound infection with no antibiotics?"
```

What Happens:

1. OllamaBackend is created (connects to local Ollama at port 11434).
2. ContextRetriever loads the FAISS index from `data/vector_store/`.
3. RAGPipeline.query() is called:
   - Classifier extracts sub-queries: `["treat deep wound infection", "antibiotics"]`
   - Each sub-query is embedded and searched against FAISS.
   - Results are deduplicated and capped.
   - Classifier determines: `mode=SURVIVAL, urgency=HIGH, resources=LIMITED`.
   - The system prompt is assembled with the retrieved chunks and classification flags.
   - The full prompt is sent to Ollama (`qwen2.5:3b`).
   - The response streams to the terminal, token by token.

Terminal Output Example:

```
──────────────────────────────────────────────────
GridMind Query: How do I treat a deep wound infection...
──────────────────────────────────────────────────
[*] Retrieving context chunks from local vector store...
[*] Decomposed query into 2 search vectors.
[*] Context retrieved! Prompt length is 3254 characters.
[*] Classifier detected: mode: SURVIVAL | resources: LIMITED | urgency: HIGH
[*] LLM analyzing context...

**Do this:**
1. Clean the wound immediately using boiled water...
2. Create a honey/sugar poultice...
...
**Hard stops:**
```

```
- Do NOT close an infected wound with sutures...
...
[SYS] Output complete.
```

Step 9.2 — Domain-Filtered Query

If you know the topic area, you can filter results to a specific domain for higher precision:

```
python main.py ask "How to set a broken bone" --domain 02_health
```

Step 9.3 — Adjusting Context Depth

More context = more data for the LLM, but slower and uses more tokens:

```
python main.py ask "water filtration methods" --top-k 5
```

Step 9.4 — Raw LLM Test (No RAG)

Test the LLM directly without any knowledge retrieval:

```
python main.py llm --backend ollama
```

This sends a hardcoded prompt ("What are the three most important things to do if you are lost in the wilderness?") and streams the response. Useful for verifying the LLM is working correctly.

Step 9.5 — Interactive Loop (Custom Script)

For continuous use, create a simple wrapper script at `/home/gridmind/OFFGRIDMIND/run.sh`:

```
#!/bin/bash
cd /home/gridmind/OFFGRIDMIND
source .venv/bin/activate
echo "=========================================="
echo "  GRIDMIND TERMINAL — Offline Survival AI"
echo "  Type your question and press Enter."
echo "  Type 'exit' to quit."
echo "=========================================="
while true; do
    echo ""
    read -p "GRIDMIND> " query
    if [ "$query" == "exit" ]; then
        echo "System offline."
```

```
        break
    fi
    python main.py ask "$query" --top-k 3
done
```

Make it executable and run:

```
chmod +x /home/gridmind/OFFGRIDMIND/run.sh
./run.sh
```

CHAPTER 10: END-TO-END QUERY LIFECYCLE

This chapter traces the exact path of a single query from the moment the user types it to the moment the answer appears on screen.

User types: `"how to start a fire with no matches in wet conditions"`

STEP	COMPONENT	ACTION
1	`main.py`	Parses the command. Subcommand = "ask". Calls `run_ask_test(args)`.
2	`llm_engine.py`	`create_llm_backend("ollama")` returns OllamaBackend instance. Connects to `http://localhost:11434`.
3	`retriever.py`	ContextRetriever is created. `load_index()` reads `data/vector_store/index.faiss` and `metadata.json`.
4a	`classifier.py`	`extract_search_queries()`: Splits by punctuation → removes stop-words → returns `["start fire matches wet conditions"]`.
4b	`classifier.py`	`classify()`: mode_survival matches "start fire". resources_limited matches "no matches". Result: `{mode: SURVIVAL, resources: LIMITED, urgency: LOW}`.
4c	`retriever.py`	`retrieve("start fire matches wet conditions")`: Embeds query → FAISS finds 3 nearest neighbors → Returns 3 chunks about fire-starting from `raw_docs/06_fire/`.
4d	`rag_pipeline.py`	Deduplication removes identical chunks. Prompt is assembled: system_prompt + retrieved chunks + classification flags + user question.
4e	`llm_engine.py`	`OllamaBackend.generate(final_prompt, stream=True)`: POSTs to `/api/generate`. Streams response line-by-line. Each JSON line yields 1 token.
5	Terminal	Tokens are printed in real-time. Total time: 30-120 seconds depending on response length.

CHAPTER 11: PERFORMANCE BENCHMARKS & EXPECTATIONS

Raspberry Pi 5 (8GB) — Qwen 2.5 3B (Q4_K_M via Ollama)

METRIC	VALUE
Model Load Time (Cold Start)	4-6 seconds
Prompt Evaluation Speed	~20-30 tokens/sec
Text Generation Speed	~4-7 tokens/sec
Typical Full Response Time	30-120 seconds
Steady-State RAM Usage	~3.0 GB
Peak RAM Usage (during gen)	~4.5 GB
CPU Utilization (during gen)	100% all 4 cores
CPU Temperature (active cooling)	55-65°C
FAISS Search Time (10k chunks)	~10-15 ms
Embedding Time (single query)	~200-400 ms

Raspberry Pi 4 (8GB) — Same Configuration

METRIC	VALUE
Model Load Time (Cold Start)	8-12 seconds
Prompt Evaluation Speed	~10-15 tokens/sec
Text Generation Speed	~2-4 tokens/sec
Typical Full Response Time	60-180 seconds
Steady-State RAM Usage	~3.0 GB
Peak RAM Usage (during gen)	~4.5 GB
CPU Temperature (active cooling)	60-72°C

> **NOTE:** The Pi 5 is approximately 2× faster than the Pi 4 for LLM inference due to its upgraded

> Cortex-A76 cores. The Pi 5 is strongly recommended for a tolerable user experience.

CHAPTER 12: MAINTENANCE, BACKUP, & RECOVERY

Step 12.1 — Backing Up the Knowledge Base

The critical files to back up are:

#	PATH	CONTENTS
1	`raw_docs/`	Your source documents (PDFs, TXTs, MDs)
2	`data/vector_store/`	The FAISS index + metadata
3	`prompts/system_prompt.txt`	Your customized AI persona

Copy to a USB drive:

```
cp -r raw_docs/ /mnt/usb/gridmind_backup/
cp -r data/vector_store/ /mnt/usb/gridmind_backup/
```

Step 12.2 — Recovering from Index Corruption

If `data/vector_store/index.faiss` becomes corrupted:

1. Delete the vector_store directory:

   ```
   rm -rf data/vector_store/
   ```

2. Rebuild from scratch:

   ```
   python main.py index --batch-size 16
   ```

The `raw_docs/` are always the source of truth. As long as the source documents exist, the index can be rebuilt.

Step 12.3 — Auto-Start GridMind on Boot

For a truly standalone device, configure GridMind to start automatically when the Pi boots.

1. Enable Ollama auto-start:

```
sudo systemctl enable ollama
```

2. Create an auto-start script:

```
sudo nano /etc/profile.d/gridmind_autostart.sh
```

Contents:

```
#!/bin/bash
if [ "$(tty)" = "/dev/tty1" ]; then
    cd /home/gridmind/OFFGRIDMIND
    source .venv/bin/activate
    ./run.sh
fi
```

3. Enable auto-login on tty1:

```
sudo raspi-config
# → System Options → Boot / Auto Login → Console Autologin
```

Boot Sequence:

1. OS starts in ~15 seconds.
2. Auto-login as 'gridmind'.
3. Ollama service starts automatically.
4. `run.sh` launches the GridMind terminal.
5. User sees `GRIDMIND>` prompt and can start querying.

CHAPTER 13: FINAL PRODUCT — WHAT YOU HAVE BUILT

At the end of this manual, you have constructed a fully standalone survival intelligence terminal:

CATEGORY	DETAIL
Hardware	Raspberry Pi 5 (8GB) + Active Cooling + 64GB microSD. Optional: External SD card reader for knowledge expansion. Optional: 20,000mAh USB battery pack for field deployment.
Operating System	Raspberry Pi OS Lite (64-bit, Headless)
LLM Server	Ollama (Local Inference)
LLM Model	Qwen 2.5 3B Q4_K_M (1.9GB Survival Brain)
Embedding Model	nomic-embed-text (768-dim Semantic Search)
Vector Store	FAISS IndexFlatL2 (Exact Vector Search)
Application	GridMind RAG Pipeline (Python, Terminal-Only)
Network	ZERO. Fully Air-Gapped.
Internet	NEVER required again.

Capabilities

- Answer survival questions across 15+ domains.
- Medical, water, food, fire, shelter, navigation, defense, electronics, and more.
- Urgency-aware response generation.
- Resource-adaptive recommendations.
- Expandable knowledge base via SD card.
- Fully offline, fully air-gapped.
- Boots directly into query terminal.
- Powered by battery for field deployment.

APPENDIX A: QUICK-START CHEAT SHEET

After full setup is complete, the daily workflow is:

Power On

1. Plug in power.
2. Wait 30 seconds.
3. Terminal appears with `GRIDMIND>` prompt.

Ask a Question

```
GRIDMIND> How do I set a broken bone with no splint material?
# → Answer streams to screen in 30-120 seconds.
```

Add New Knowledge

```
# 1. Insert SD card with USB reader.
sudo mount /dev/sda1 /mnt/knowledge
# 2. Copy files.
cp -r /mnt/knowledge/* raw_docs/
# 3. Index new files.
python main.py index --incremental --batch-size 16
# 4. Unmount.
sudo umount /mnt/knowledge
# New knowledge is now searchable.
```

Full Re-Index (if needed)

```
python main.py index --batch-size 16
```

Test LLM Health

```
python main.py llm
```

APPENDIX B: TROUBLESHOOTING

PROBLEM	SOLUTION
"Ollama health check failed"	`sudo systemctl restart ollama` `ollama list   # Verify models are present`
Out of Memory (OOM) during indexing	Use smaller batch size: `python main.py index --batch-size 8`. Ensure no other processes are running: `sudo systemctl stop cron`.
Extremely slow generation (<1 token/sec)	Verify cooling is working. Check CPU frequency: `cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq` (should be 2400000 for Pi 5). Check throttling: `vcgencmd get_throttled` (0x0 = not throttled).
Model not found in Ollama	You need internet temporarily to pull the model: `ollama pull qwen2.5:3b` and `ollama pull nomic-embed-text`. Then disconnect from internet again.
"No documents found" during indexing	Verify raw_docs/ contains files: `find raw_docs/ -type f | head -20`. Only .pdf, .txt, and .md files are supported.
FAISS search returns no results	Index may be empty or corrupted. Rebuild: `rm -rf data/vector_store/ && python main.py index --batch-size 16`.
Pi overheating and throttling	Install a better heatsink/fan combo. Ensure ventilation in the case. In `/boot/firmware/config.txt`, set `over_voltage=4` (Pi 4). Lower ambient temperature (shade, elevation).

EOF